

Представьте себе архитектуру, которая легла в основу современного Интернета и позволила ему вырасти до невероятных масштабов, — это и есть Representational State Transfer, или REST. Этот термин был введен Роем Томасом Филдингом в его диссертации 2000 года, где он проанализировал архитектуру Всемирной паутины и обобщил ее в виде архитектурного стиля для сетевых приложений. Важно понимать, что REST — это не набор готовых библиотек или протоколов, а скорее набор архитектурных принципов и ограничений, которые, будучи примененными к системе, наделяют ее желаемыми свойствами, такими как масштабируемость, простота, надежность и возможность независимого развития компонентов. Филдинг вывел этот стиль, начав с так называемого "нулевого стиля" — системы без каких-либо ограничений — и затем последовательно добавляя к нему ключевые архитектурные ограничения. Каждое из этих добавлений решало определенную проблему и формировало облик того, что мы сегодня знаем как REST.

Первым и фундаментальным ограничением является разделение на клиент-серверную архитектуру. Этот принцип разделяет ответственность: пользовательский интерфейс и все, что с ним связано, является заботой клиента, в то время как хранение и управление данными — это прерогатива сервера. Такое разделение обязанностей приносит множество выгод: оно повышает переносимость пользовательского интерфейса на разные платформы, упрощает серверные компоненты, что улучшает масштабируемость, и, что самое важное для Интернета, позволяет клиентской и серверной частям эволюционировать независимо друг от друга.

5.1.2 Client-Server

The first constraints added to our hybrid style are those of the client-server architectural style (Figure 5-2), described in Section 3.4.1. Separation of concerns is the principle behind the client-server constraints. By separating the user interface concerns from the data storage concerns, we improve the portability of the user interface across multiple platforms and improve scalability by simplifying the server components. Perhaps most significant to the Web, however, is that the separation allows the components to evolve independently, thus supporting the Internet-scale requirement of multiple organizational domains.



Figure 5-2. Client-Server

Следующим, и одним из самых важных, ограничением является требование отсутствия состояния (stateless) на стороне сервера. В рамках этого ограничения каждый запрос от клиента к серверу должен содержать всю информацию, необходимую для его понимания, и не может использовать какой-либо сохраненный на сервере контекст, а сессионное состояние полностью хранится на клиенте. Это кардинально меняет взаимодействие: сервер не помнит предыдущие запросы клиента, каждый запрос обрабатывается изолированно. Именно это ограничение обеспечивает такие свойства, как видимость (так как для понимания запроса не нужно смотреть ни на что, кроме самого запроса), надежность (так как упрощается восстановление после частичных сбоев) и, что критически важно, масштабируемость. Серверы в такой архитектуре становятся проще и могут легко обслуживать огромное количество клиентов, не тратя ресурсы на хранение состояния каждого из них.

5.1.3 Stateless

We next add a constraint to the client-server interaction: communication must be stateless in nature, as in the client-stateless-server (CSS) style of Section 3.4.3 (Figure 5-3), such that each request from client to server must contain all of the information necessary to understand the request, and cannot take advantage of any stored context on the server. Session state is therefore kept entirely on the client.

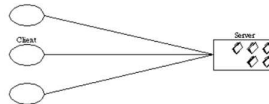


Figure 5-3. Client-Stateless-Server

This constraint induces the properties of visibility, reliability, and scalability. Visibility is improved because a monitoring system does not have to look beyond a single request datum in order to determine the full nature of the request. Reliability is improved because it eases the task of recovering from partial failures [133]. Scalability is improved because not having to store state between requests allows the server component to quickly free resources, and further simplifies implementation because the server doesn't have to manage resource usage across requests.

Like most architectural choices, the stateless constraint reflects a design trade-off. The disadvantage is that it may decrease network performance by increasing the repetitive data (per-interaction overhead) sent in a series of requests, since that data cannot be left on the server in a shared context. In addition, placing the application state on the client-side reduces the server's control over consistent application behavior, since the application becomes dependent on the correct implementation of semantics across multiple client versions.

Центральным понятием в REST является "ресурс" (resource). Филдинг определяет ресурс как концептуальное отображение на некоторую информацию, а не как саму информацию. Каждый ресурс в системе идентифицируется с помощью универсального идентификатора ресурса (URI). Однако сам ресурс — это абстракция, а клиенты взаимодействуют не с ним напрямую, а с его представлениями (representations). Представление — это, по сути, снимок состояния ресурса в определенный момент времени, переданный в некотором формате, например, в HTML, JSON или XML. Именно эта идея передачи состояния ресурса через его представления и дала название всему стилю — Representational State Transfer.

5.2.1.1 Resources and Resource Identifiers

The key abstraction of information in REST is a *resource*. Any information that can be named can be a resource: a document or image, a temporal service (e.g. "today's weather in Los Angeles"), a collection of other resources, a non-virtual object (e.g. a person), and so on. In other words, any concept that might be the target of an author's hypertext reference must fit within the definition of a resource. A resource is a conceptual mapping to a set of entities, not the entity that corresponds to the mapping at any particular point in time.

More precisely, a resource R is a temporally varying membership function $M_R(t)$, which for time t maps to a set of entities, or values, which are equivalent. The values in the set may be *resource representations* and/or *resource identifiers*. A resource can map to the empty set, which allows references to be made to a concept before any realization of that concept exists -- a notion that was foreign to most hypertext systems prior to the Web [61]. Some resources are static in the sense that, when examined at any time after their creation, they always correspond to the same value set. Others have a high degree of variance in their value over time. The only thing that is required to be static for a resource is the semantics of the mapping, since the semantics is what distinguishes one resource from another.

Взаимодействие между клиентом и сервером строится на основе единообразного интерфейса (uniform interface). Это ограничение является краеугольным камнем REST и, возможно, самым сложным для понимания. Оно подразумевает, что все взаимодействия между компонентами должны следовать одним и тем же правилам, что делает архитектуру простой, видимой и разделяемой. Единообразный интерфейс REST, как правило, реализуется через методы протокола HTTP, которые образуют своего рода глаголы для работы с ресурсами: GET для получения представления ресурса, POST для создания нового ресурса, PUT для полного обновления существующего и DELETE для его удаления. Это означает, что, зная URI ресурса и используя стандартные HTTP-методы, любой клиент может взаимодействовать с ним предсказуемым образом, без необходимости знать что-либо о внутренней структуре сервера. Также важной частью этого интерфейса является то, что сообщения являются самодостаточными: они содержат всю информацию, необходимую для их обработки, включая метаданные о том, как интерпретировать представление.

5.1.5 Uniform Interface

The central feature that distinguishes the REST architectural style from other network-based styles is its emphasis on a uniform interface between components (Figure 5-6). By applying the software engineering principle of generality to the component interface, the overall system architecture is simplified and the visibility of interactions is improved. Implementations are decoupled from the services they provide, which encourages independent evolvability. The trade-off, though, is that a uniform interface degrades efficiency, since information is transferred in a standardized form rather than one which is specific to an application's needs. The REST interface is designed to be efficient for large-grain hypermedia data transfer, optimizing for the common case of the Web, but resulting in an interface that is not optimal for other forms of architectural interaction.

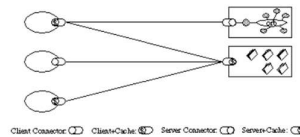


Figure 5-6. Uniform-Client-Cache- Stateless-Server

In order to obtain a uniform interface, multiple architectural constraints are needed to guide the behavior of components. REST is defined by four interface constraints: identification of resources; manipulation of resources through representations; self-descriptive messages; and, hypermedia as the engine of application state. These constraints will be discussed in Section 5.2.

Помимо этих основных ограничений, REST включает и другие, которые в совокупности и создают его мощь. Кэшируемость (cacheable) позволяет клиентам и промежуточным узлам сохранять ответы сервера, чтобы использовать их для последующих эквивалентных запросов, что значительно повышает эффективность и снижает нагрузку на сеть.

5.1.4 Cache

In order to improve network efficiency, we add cache constraints to form the client-cache-stateless-server style of Section 3.4.4 (Figure 5-4). Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable. If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.

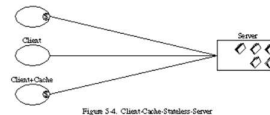


Figure 5-4. Client-Cache- Stateless-Server

The advantage of adding cache constraints is that they have the potential to partially or completely eliminate some interactions, improving efficiency, scalability, and user-perceived performance by reducing the average latency of a series of interactions. The trade-off, however, is that a cache can decrease reliability if stale data within the cache differs significantly from the data that would have been obtained had the request been sent directly to the server.

Многоуровневая система (layered system) ограничивает поведение компонентов таким образом, что архитектура может быть организована в иерархические слои. Это позволяет, например, вводить между клиентом и сервером промежуточные узлы, такие как прокси-серверы или балансировщики нагрузки, которые могут предоставлять дополнительные функции, оставаясь невидимыми для клиента.

5.1.6 Layered System

In order to further improve behavior for Internet-scale requirements, we add layered system constraints (Figure 5-7). As described in Section 3.4.2, the layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component cannot "see" beyond the immediate layer with which they are interacting. By restricting knowledge of the system to a single layer, we place a bound on the overall system complexity and promote substrate independence. Layers can be used to encapsulate legacy services and to protect new services from legacy clients, simplifying components by moving infrequently used functionality to a shared intermediary. Intermediaries can also be used to improve system scalability by enabling load balancing of services across multiple networks and processors.

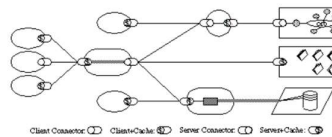


Figure 5-7. Uniform-Layered-Client-Cache- Stateless-Server

The primary disadvantage of layered systems is that they add overhead and latency to the processing of data, reducing user-perceived performance [32]. For a network-based system that supports cache constraints, this can be offset by the benefits of shared caching at intermediaries. Placing shared caches at the boundaries of an organizational domain can result in significant performance benefits [136]. Such layers also allow security policies to be enforced on data crossing the organizational boundary, as is required by firewalls [79].

The combination of layered system and uniform interface constraints induces architectural properties similar to those of the uniform pipe-and-filter style (Section 3.2.2). Although REST interaction is two-way, the large-grain data flows of hypermedia interaction can each be processed like a data-flow network, with filter components selectively applied to the data stream in order to transform the content as it passes [26]. Within REST, intermediary components can actively transform the content of messages because the messages are self-descriptive and their semantics are visible to intermediaries.

И, наконец, опциональное ограничение "код по требованию" (code on demand) позволяет серверу временно расширять функциональность клиента, передавая ему исполняемый код, например, скрипты на JavaScript.

5.1.7 Code-On-Demand

The final addition to our constraint set for REST comes from the code-on-demand style of Section 3.5.3 (Figure 5-8). REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts. This simplifies clients by reducing the number of features required to be pre-implemented. Allowing features to be downloaded after deployment improves system extensibility. However, it also reduces visibility, and thus is only an optional constraint within REST.

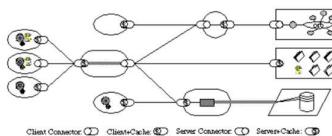


Figure 5-8. REST

The notion of an optional constraint may seem like an oxymoron. However, it does have a purpose in the architectural design of a system that encompasses multiple organizational boundaries. It means that the architecture only gains the benefit (and suffers the disadvantages) of the optional constraints when they are known to be in effect for some realm of the overall system. For example, if all of the client software within an organization is known to support Java applets [43], then services within that organization can be constructed such that they gain the benefit of enhanced functionality via downloadable Java classes. At the same time, however, the organization's firewall may prevent the transfer of Java applets from external sources, and thus to the rest of the Web it will appear as if those clients do not support code-on-demand. An optional constraint allows us to design an architecture that supports the desired behavior in the general case, but with the understanding that it may be disabled within some contexts.

Теперь, понимая теоретические основы REST, заложенные в диссертации Филдинга, мы можем увидеть, как этот архитектурный стиль становится идеальным инструментом для создания и взаимодействия с динамическими сайтами. В контексте традиционного веб-сайта сервер генерирует полные HTML-страницы и отправляет их клиенту, который просто их отображает.

Такой подход является монолитным и менее гибким. REST API предлагает совершенно иную модель, которая лежит в основе архитектуры современных динамических веб-приложений, часто называемых односторонними приложениями (SPA). В этой модели сервер выступает в роли поставщика данных и бизнес-логики, предоставляя REST API как набор конечных точек (endpoints) для доступа к ресурсам. Клиентская же часть, написанная, например, на JavaScript с использованием фреймворков вроде React, Vue.js или Angular, берет на себя всю ответственность за пользовательский интерфейс и его динамическое обновление.

Взаимодействие между клиентом и сервером в рамках RESTful API строится по четкому и стандартизированному сценарию, который во многом повторяет принципы работы самого Интернета. Базовый цикл начинается с того, что клиент, будь то человек или программная система, инициирует запрос к серверу, когда ему требуется доступ к какому-либо ресурсу. Руководствуясь документацией API, клиент форматирует этот запрос таким образом, чтобы он был понятен серверу. Такой запрос всегда содержит несколько ключевых компонентов. Прежде всего, это уникальный идентификатор ресурса, в роли которого для REST-сервисов выступает универсальный указатель ресурсов (URL), четко указывающий серверу, что именно требуется клиенту. Далее, запрос обязательно включает метод, который сообщает серверу, какое действие необходимо выполнить с ресурсом. Эти методы, как правило, реализуются через протокол HTTP, и среди них выделяются четыре основных: GET для доступа к ресурсам, POST для отправки новых данных на сервер (с учетом того, что многократная отправка одного и того же POST-запроса может привести к созданию нескольких ресурсов), PUT для обновления существующих ресурсов (отправка одного и того же PUT-запроса несколько раз дает один и тот же результат) и DELETE для удаления ресурса. Помимо URL и метода, запрос содержит заголовки HTTP — метаданные, в которых, например, указывается формат запроса и ответа, а также сами данные, необходимые для успешной работы методов POST и PUT. Наконец, запрос может включать различные параметры — например, параметры пути, уточняющие URL, или параметры запроса, которые предоставляют серверу более детальную информацию о необходимых действиях.

Метод
Как правило, разработчики реализуют RESTful API с помощью протокола передачи гипертекста (HTTP). Метод HTTP сообщает серверу, что ему необходимо сделать с ресурсом. Ниже приведены четыре распространенных метода HTTP:
GET
Клиенты используют GET для доступа к ресурсам, расположенным на сервере по указанному URL. Они могут кэшировать запросы GET и отправлять параметры в запросе RESTful API, чтобы сообщить серверу о необходимости фильтровать данные перед отправкой.
POST
Клиенты используют POST для отправки данных на сервер. При этом они включают в запрос представления данных. Отправка одного и того же запроса POST несколько раз имеет побочный эффект — многократное создание одного и того же ресурса.
PUT
Клиенты используют PUT для обновления существующих на сервере ресурсов. В отличие от POST, отправка одного и того же запроса PUT несколько раз дает один и тот же результат в веб-службе RESTful.
DELETE
Клиенты используют запрос DELETE для удаления ресурса. Запрос DELETE может изменить состояние сервера. Однако если у пользователя нет соответствующей аутентификации, запрос завершается ошибкой.

представление запрошенного ресурса. Сервер выбирает подходящий формат этого представления, например JSON или XML, на основе заголовков запроса.

Что содержит ответ сервера RESTful API?

Принципы REST требуют, чтобы ответ сервера содержал следующие компоненты:
Строка состояния
Строка состояния содержит трехзначный код состояния, который сообщает об успешном или неудачном выполнении запроса. Например, коды 2XX указывают на успешное выполнение, а коды 4XX и 5XX — на ошибки. Коды 3XX указывают на перенаправление URL.
Ниже приведены некоторые распространенные коды состояния:
• 200: общий ответ об успешном выполнении • 201: ответ об успешном выполнении метода POST • 400: неверный запрос, который сервер не может обработать • 404: ресурс не найден
Текст сообщения
Текст ответа содержит представление ресурса. Сервер выбирает подходящий формат представления на основе содержания заголовков запроса. Клиенты могут запрашивать информацию в форматах XML или JSON: они определяют запись данных в виде обычного текста. Например, если клиент запрашивает имя и возраст человека по имени Джон, сервер возвращает представление JSON в следующем формате:
<pre>{ "name": "John", "age": 30 }</pre>
Заголовки
Ответ также содержит заголовки или метаданные об ответе. Они дают более подробный контекст ответа и включают такую информацию, как название сервера, кодировка, дата и тип контента.

Получив запрос, сервер проходит через несколько обязательных этапов. Прежде всего, он аутентифицирует клиента, чтобы подтвердить его личность и удостовериться, что тот имеет право сделать данный запрос. Для этого RESTful API поддерживает различные методы аутентификации: от базовой HTTP-аутентификации с передачей имени пользователя и пароля в заголовке до более сложных схем, таких как аутентификация носителя с использованием токенов, ключи API или протокол OAuth. После успешной аутентификации сервер получает запрос и внутренне обрабатывает его. Завершающим этапом является возврат ответа клиенту. Этот ответ, в свою очередь, также структурирован в соответствии с принципами REST и содержит строку состояния с трехзначным кодом, который информирует об успешном или неудачном выполнении запроса (например, коды 2XX означают успех, а 4XX и 5XX — ошибки). Кроме того, ответ содержит заголовки с метаданными, такими как тип контента или дата, и, что самое важное, тело сообщения, которое содержит

Именно эта строгая и единообразная структура запросов и ответов, построенная на принципах REST, и позволяет динамическим сайтам функционировать как полноценные приложения, обмениваясь данными с сервером в реальном времени без необходимости полной перезагрузки страницы. Более того, поскольку REST API не зависит от используемой технологии, клиентские и серверные приложения могут быть написаны на совершенно разных языках программирования, и изменение базовой технологии на любой из сторон не повлияет на обмен данными. Это разделение клиента и сервера упрощает и разделяет различные компоненты, позволяя каждой части развиваться независимо: например, разработчики могут вносить изменения в уровень базы данных, не переписывая логику приложения, а изменения платформы или технологии на сервере не влияют на клиентское приложение. Такая гибкость, масштабируемость и независимость, подкрепленные преимуществами отсутствия состояния и кэширования, делают RESTful API идеальным инструментом для создания современных, интерактивных и высокопроизводительных веб-решений.

Источники:

https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

<https://aws.amazon.com/ru/what-is/restful-api/>